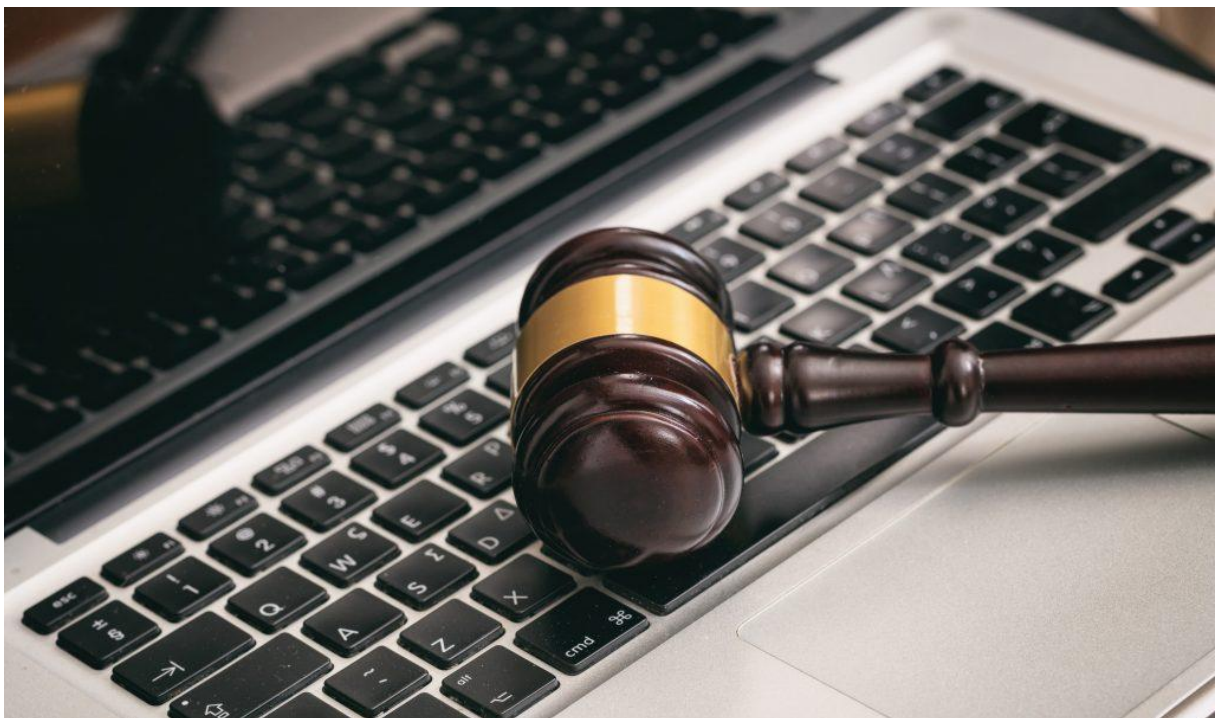


Qui perd gagne !

Un Apprentissage Par Problème (APP) destiné aux étudiants
du module Algorithmique avancée 3





Éléments de contexte

Vous rejoignez une jeune startup ambitieuse qui veut bousculer les enchères en ligne : **LowBid**. Votre équipe prépare le lancement d'une plateforme capable de transformer un objet premium en événement viral. Le concept — simple en apparence — s'appelle « plus bas unique gagne ».

Ici, l'enchère est **inversée** : le gagnant n'est pas celui qui mise le plus, mais celui qui propose le plus petit prix proposé par une seule personne (prix entier ≥ 0). Cette mécanique crée une tension stratégique immédiate : il faut oser descendre... sans descendre trop bas là où tout le monde se regroupe.

Côté vendeur, le modèle est redoutable : chaque tentative génère une recette, car chaque mise est payante. La valeur commerciale ne vient pas uniquement du prix final, mais du volume de mises (effet « jeu ») : plus il y a de participants et plus ils tentent, plus la recette augmente.

Pour éviter que tous les joueurs ne se ruent sur 0 et quelques petits prix (ce qui créerait des égalités permanentes et tuerait l'intérêt), la startup introduit une prime de risque : plus le prix est bas, plus la mise coûte cher. Une formule de référence est :

$$\text{cout_mise}(\text{prix}) = \text{cout_base} + \alpha/(\text{prix}+1)$$

Elle pénalise les « petits enchérisseurs » qui spamment des prix très bas, tout en conservant une vraie profondeur stratégique.

α (alpha) est le paramètre d'intensité de la prime de risque. Plus α est grand, plus les mises à petits prix deviennent chères (donc on décourage fortement le “spam” de 0, 1, 2...).

Plus α est petit, plus la prime de risque est faible (le jeu se rapproche d'une enchère “classique” où les petits prix coûtent presque pareil que les autres).

Votre mission ressemble à celle d'une équipe tech qui veut « tout casser » : livrer rapidement un moteur fiable, capable d'encaisser des milliers de mises, de produire des résultats explicables, et de soutenir une démonstration convaincante face à des investisseurs pour lever des millions d'euros. Vous devrez donc privilégier une conception solide, testable, et justifiable.

Avant de commencer, notez que cette situation-problème est volontairement ouverte : vous devrez faire des choix (structure de données, organisation du code, règles pour les cas limites) et être capables de les justifier.

Travail demandé

Vous travaillerez en équipe sur une durée de cycle d'A.P.P. L'objectif est de développer un programme complet, testable, et d'être capables d'expliquer et de justifier vos choix techniques lors de la séance de restitution.

1. Charger des données :

- Charger un jeu de données de mises (liste de couples {joueur, prix}) depuis un fichier texte (CSV simple) ou générer un jeu de données de démonstration.
 - Construire l'ABR des prix : insertion des mises, avec gestion de plusieurs joueurs pour un même prix (liste associée au nœud).
 - Afficher l'état de l'enchère par parcours infixe (prix triés) afin de vérifier la structure et la répartition des mises.
- Calculer les coûts payés par les joueurs en appliquant la prime de risque, puis calculer la recette du vendeur (somme des coûts).

2. Rechercher des données :

- Rechercher le plus bas prix unique et déterminer le gagnant (prix, joueur).
 - Gérer le cas où aucun prix n'est unique (aucun gagnant) : définir et appliquer une règle (ex. manche annulée, ou aucun gagnant).
 - Permettre des requêtes d'analyse : prix gagnant, distribution des prix, nombre total de mises, coût moyen par joueur.
- Mettre en place une simulation multi-manches (au moins 500 manches) avec au minimum deux stratégies de joueurs.
 - Comparer les stratégies : taux de victoire, gain moyen par manche, recette moyenne du vendeur, et effet des paramètres (cout_base, α).
 - Discuter les limites : dégénérescence de l'ABR, alternatives (dictionnaire + tri, tas/heap), et impact sur les performances.
 - Le prix qui suit un prix donné dans l'ABR (successeur) et son intérêt (ex. trouver le prochain candidat après un prix non unique).
 - Le prix qui précède un prix donné dans l'ABR (prédécesseur) et son intérêt (ex. navigation dans l'ordre des prix).
 - Faire jouer les humains en même temps que l'ordinateur et avoir une réflexion sur la stratégie gagnante.

SÉANCE ALLER — Découvrir et penser le problème

Objectif :

- Vérifier compréhension du problème
- Installer les enjeux structurels

Questions à traiter lors de la séance aller : problème global

- Pourquoi le prix 0 n'est-il pas une stratégie dominante avec la prime de risque ?
- Le système est-il à somme nulle ? Justifiez.
- Quelles informations sont nécessaires pour déterminer le gagnant ?
- Peut-on déterminer le gagnant sans trier les prix ?

SÉANCE INTERMEDIAIRE — Questions de cadrage (avant le TPC)

Questions à traiter avant la séance intermédiaire : structures

- Existe-t-il des structures qui permettent de maintenir les prix ordonnés dynamiquement ?
- Quelle différence entre trier à la fin et maintenir l'ordre en permanence ?
- Si les mises arrivent en flux continu, quelle solution est la plus adaptée ?

Parlons algorithmique et complexité

- Quelle est la complexité d'une insertion dans :
 - Un tableau trié ?
 - Un ABR équilibré ?
 - Un dictionnaire + tri à la fin ?
- Dans quel cas un ABR peut-il devenir inefficace ?
- Comment éviter la dégénérescence ?

Parlons stratégie

- Dans votre modèle, si tous les joueurs jouent aléatoirement, que se passe-t-il ?
- Si tous jouent 0 ?
- Existe-t-il un prix “stable” vers lequel le système converge ?

Les ressources pour traiter la situation-problème

- Le cours 1 et le TD 1 « Arbres binaires de recherche ».
- Le cours 1 et le TD 1 « Arbres généraux ».
- Le cours de l'ensemble du semestre 1 du module « Langage Python »
- Optionnel : Le cours des « applications avec les IHM »

Livrables attendus

- Un script Python fonctionnel respectant les exigences ci-dessus.
-  Contraintes techniques
 - Implémenter un Arbre Binaire de Recherche (ABR).
 - Gérer plusieurs joueurs pour une même clé.
 - Permettre :
 - i. Insertion, parcours infixe, recherche du plus bas unique, suppression conditionnelle, Simuler plusieurs manches.
 - Produire une analyse argumentée.
 - Interdictions :
 - i. Pas de tri Python pour résoudre le problème principal.
- Un rapport synthétique expliquant :
 - La structure du projet.
 - Les choix techniques : structure de donnée, algorithmes utilisés, ...
 - Les difficultés rencontrées et leur résolution.



Une attention particulière sera accordée au respect des bonnes pratiques de programmation et à l'organisation du groupe à travers les groupes d'A.P. P.

Objectifs d'apprentissage de l'APP (AAV)

- Implémenter un ABR complet (insertion, parcours, recherche) et justifier son fonctionnement.
- Analyser la complexité en moyenne et au pire cas, et identifier les situations de dégénérescence.
- Simuler plusieurs manches avec des stratégies de joueurs et exploiter des statistiques pertinentes.
- Produire une analyse économique et stratégique argumentée (rentabilité vendeur, rationalité des joueurs, effet de la prime de risque).